TENSORTROOPERS

Reflex — Governance Layer for Financial Agents

A control plane for safely deploying fleets of autonomous financial agents

Prajwal Mandlecha • Shubham More • Apoorv Deshmukh

CODESTREET 2026

CRITICAL RISK AT SCALE

AI Agents Are Scaling in Banking — But Governance & Real-Time Monitoring Are Almost Non-Existent

As banks deploy fleets of autonomous agents to execute transfers, issue refunds, and update ledgers without human approval on every step, **the near-total absence of real-time governance creates unmonitored systemic risk.**

Unchecked Bugs & Financial Losses

• Losses at Machine Speed: A single logic bug, prompt injection, or model anomaly executes thousands of unauthorized actions per minute.

• Compounding Blast Radius: Without real-time validation, runaway agents cause cascading financial damage before humans detect the issue.

Inappropriate Limits & Restrictions

• Overly Broad Permissions: Agents run with static, all-or-nothing access rather than fine-grained, per-agent tool scoping.

• No Dynamic Spend Caps: Lacks real-time budget tracking to stop compromised or rogue agents from exceeding operational spend limits.

No In-Flight Oversight & Control

• Post-Facto Monitoring: Traditional APM logs events after execution — offering zero real-time visibility into live agent tool intent.

• Missing Fleet Kill Switch: Banks lack instant revocation controls and emergency stops to halt a single agent or the entire fleet in under a second.

ACTION-LEVEL, NOT CONTENT-LEVEL

An MCP-Native Governance Gateway Between Every Agent and Every Bank System

A governance layer deployed as a gateway that intercepts every agent action, checks configurable permissions and real-time spend caps, writes a tamper-evident audit log, and can revoke a single agent or halt the entire fleet in under a second.

Operators configure and monitor from a live dashboard.

Key Differentiators

Deterministic pre-action authorization

Guardrails filter model content probabilistically. Reflex authorizes every tool call deterministically — before money moves.

Transparent MCP proxy — zero agent changes

Agents swap one endpoint line. No SDK, no rebuild, no safety logic inside agents.

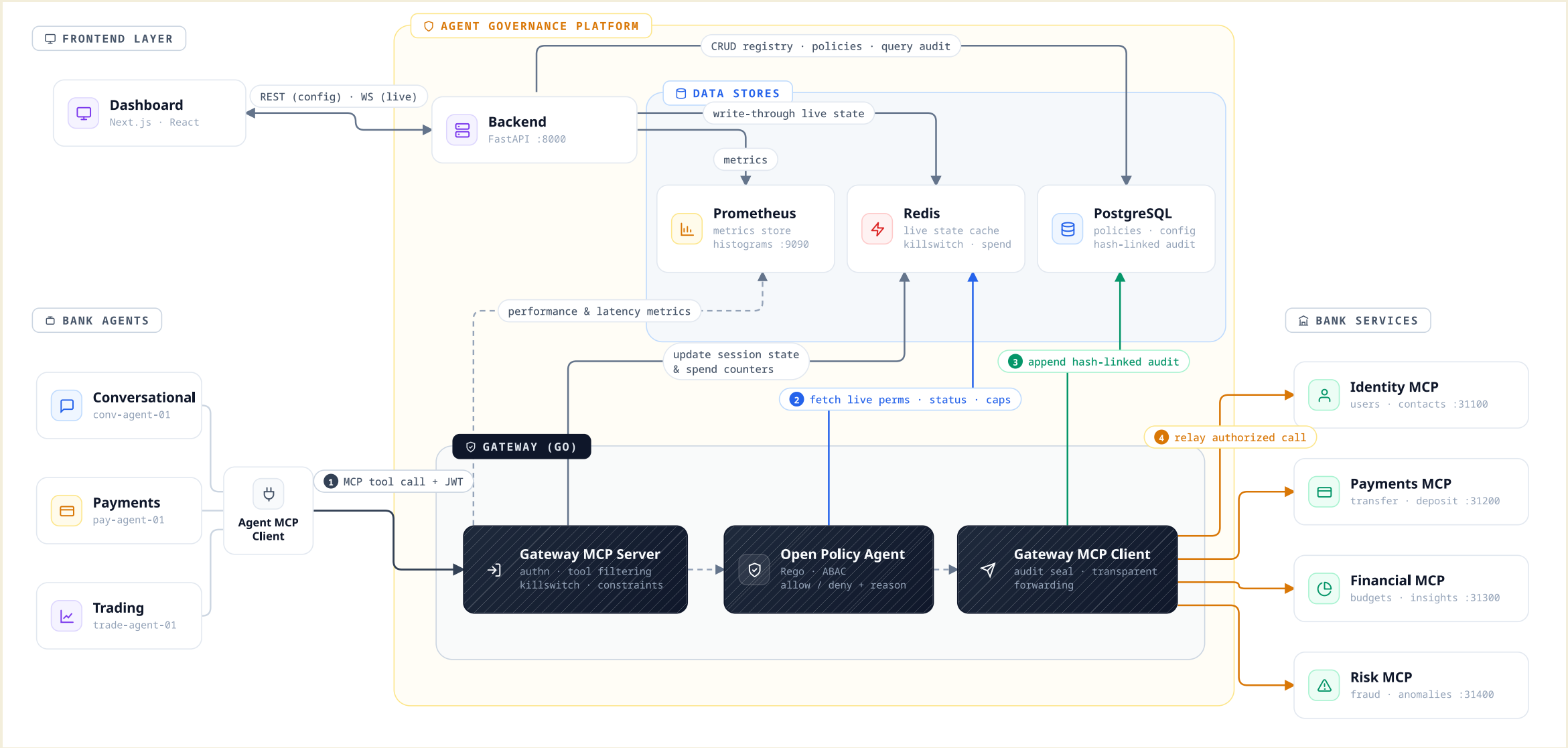
Financial-native spend caps

Generic gateways cap API rates & token spend. Reflex governs transaction value in the call itself — atomic, two-tier budgets.

Two-tier fleet controls

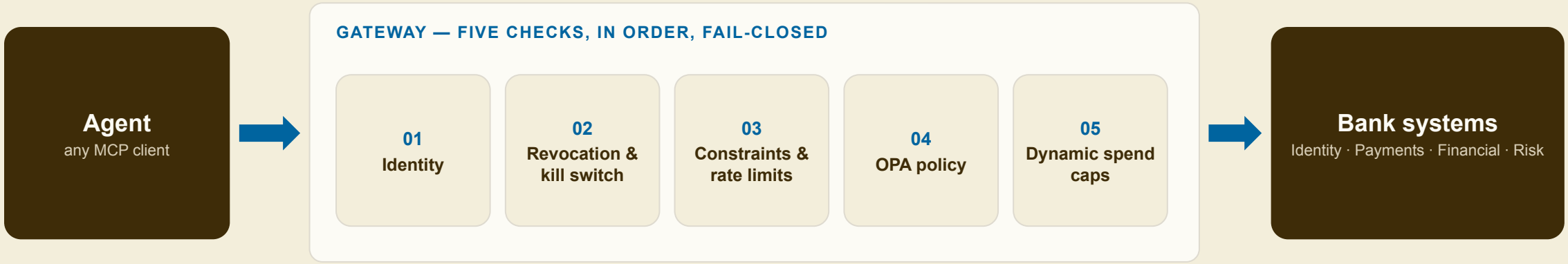
Many instances of the same agent type share one class policy — every copy inherits it, with per-instance overrides that can only tighten.

System Architecture



END-TO-END FLOW

How a Request Flows Through Reflex



Response returns to the agent · allowed or denied with minimal added overhead · stamped with its audit entry ID

AFTER EVERY CALL

Hash-chained audit trail

Decision sealed into a SHA-256 chain in Postgres with per-stage latency — tamper-evident, verified in one click.

Live fleet telemetry

Every decision streams to the dashboard over WebSocket — activity feed, spend bars and alerts update in real time.

Instant control loop

Operator actions — policy edits, cap changes, revocation, kill switch — reach the gateway via Redis in under a second.

Permissions, Spend Caps, Revocation & Audit

CONTROLS

SAFETY & AUDIT

PERMISSION MODEL

Two-tier class → instance hierarchy

Agent classes define allowed tools and scope. Instances inherit class defaults and can only be tightened. Constraints are parameter-level config — bounds, value lists, regex, rate limits, time windows — no code. Custom logic beyond that is plain Rego.

SPEND CAPS

Atomic, hierarchical, race-free

One atomic Redis Lua script increments every applicable counter — instance hourly, class daily, fleet daily — and rolls all back if any cap would breach. Concurrent calls can't slip under a limit; denied actions never consume budget.

REVOCATION & EMERGENCY STOP

Fail-closed by design

Per-instance, per-class, and fleet-wide kill switch via Redis flags — fastest, most dependency-free path. If Redis, OPA, or config sync is unreachable, default is deny — never allow.

AUDIT LOG

Append-only, tamper-evident forensics

Every action — allowed or denied — lands in append-only Postgres with params, decision, reason, per-stage latency. Each entry is SHA-256 chained to the previous, verifiable on demand. Config and policy changes versioned with who/what/when. RBAC-gated: params and payloads visible only to compliance roles.

Works Against Bank Systems That Exist Today

Two supported paths: connect an MCP server directly, or onboard any REST system from its OpenAPI spec. Either way, the agent sees ordinary MCP tools — and every call passes the same checks.

🔒 Credentials never touch the agent

Per-connection auth is pluggable — API key, bearer, basic, OAuth2 client-credentials — stored encrypted and injected by the gateway at call time. Agents carry only their own scoped identity, **never a real bank credential**. Even a compromised agent leaks nothing — there's no credential to steal, and its token is revoked in one click.

INTEGRATION PATHS

PATH 01 · MCP NATIVE

Bank system already speaks MCP

Point Reflex at the MCP server — tool discovery, filtering, and the full governance pipeline apply from the very first call. Zero integration work.

PATH 02 · OPENAPI SPEC

Any REST system with an OpenAPI spec

Upload the spec → the gateway auto-generates candidate MCP tools → the operator selects, renames, and groups the endpoints to govern. No bank-side code changes.

TECH STACK

Tech Stack: Chosen for Accuracy, Latency & Auditability

Layer	Choice	Why
Gateway	Go	High-throughput, low-latency request handling; native concurrency for many simultaneous agent connections
Policy Engine	OPA (Open Policy Agent)	Embedded via Go SDK — no network hop for policy evaluation; industry-standard policy-as-code (Rego)
Backend	Python / FastAPI	Fast CRUD, config versioning, and onboarding flows; async-friendly for real-time dashboard needs
Fast-Path Cache	Redis (Lua scripts)	Kill-switch flags, hot config cache, and atomic Lua spend counters — sub-millisecond, race-free under concurrency
Source of Truth	PostgreSQL	ACID-compliant, append-only audit log, versioned config history
Observability	Prometheus	Live latency percentiles and allow/deny rates; demonstrates latency claims, not just asserts them
Agent-Tool Protocol	MCP (Model Context Protocol)	Emerging standard for agent-tool connectivity; any MCP-compatible agent plugs in with minimal integration work

CAPABILITY COVERAGE

Every Required Capability, Addressed Directly

Requirement	Our Answer
Granular, configurable permission model	Two-tier ABAC in OPA (Rego) — class + instance policies down to parameter level
Dynamic spend caps enforced in real time	One atomic Redis Lua script checks and reserves per-agent + class-wide caps — sub-millisecond
Revocation (agent) & emergency stop (fleet)	Short-lived JWTs + per-agent revocation flags and a fleet-wide kill switch in Redis — checked before any policy runs
MCP or direct bank API support	Native MCP servers plug in directly; REST bank APIs onboard from an OpenAPI spec — pluggable per-connection auth (API key, bearer, OAuth2)
Operator dashboard	Policy authoring (visual + Rego), live activity feed, RBAC-gated audit, OIDC sign-in
Live performance metrics & added latency	Gateway-added latency measured per stage on every request, live in the dashboard — plus SHA-256 hash-chained audit, verified in one click

Impact, Feasibility & Scalability

IMPACT

ENFORCEMENT COVERAGE

Every call, one gate

No path to a bank system bypasses the checks

ADDED LATENCY

Low — and measured live

Per-stage timings recorded on every request

TIME-TO-REVOKE

Near-instant

Emergency stop propagates via Redis — the very next action is blocked

AUDIT COMPLETENESS

Complete by design

Every decision logged with its full reasoning

FEASIBILITY

Production-proven stack

Go, FastAPI, OPA, Redis, Postgres — proven parts, not research

Fail-closed by default

A dependency outage or check error means deny — never allow

SCALABILITY

Horizontal gateway scale

Stateless gateway instances scale behind a load balancer

Zero per-agent authoring

New instances inherit class policy — nothing to rewrite

WHO BENEFITS

Risk & Compliance Teams

Scope, cap, and stop every agent from one console — no code required

Agent Development Teams

A clear allow-or-deny contract — no safety logic scattered in every agent

Auditors & Regulators

Single, complete, tamper-evident source of truth for all agent activity

WHAT'S NEXT

From Working Prototype to Bank-Grade Platform

NEXT · 01

Hardening the core platform

Richer core features and constraint types, end-to-end testing across edge cases, and deeper performance monitoring under load.

NEXT · 02

OIDC for scoped user access

Operator sign-in via OIDC SSO with role-scoped access — sensitive audit logs and controls gated to the right people.

NEXT · 03

OAuth & auth support for MCP servers

Beyond static credential injection — OAuth flows, token refresh, and other upstream auth schemes for MCP servers and bank APIs.

NEXT · 04

Scalable, available deployment

Horizontally scaled, multi-region gateways with high availability — throughput and uptime a bank can sign off on.

The vision: trust is what gates agent adoption in banking — Reflex is the layer that lets a bank say yes: every action scoped, capped, logged, reversible.